计算机系统基础

Lab2 实验报告

BombLab

姓名： 吴向杰

学号： 25300190057

目录

1 实验目的 **1**

2 实验环境与要求 **1**

3 实验过程 **1**

4 各关的分析与实现 **1**

5 测试结果与验证 **8**

6 实验小结 **13**

7 参考资料与工具使用 **13**

1 实验目的

本实验通过分析个人 `bomb` 的 x86-64 汇编，恢复各关检查输入时使用的计算关系，再在 `defuser.c` 中实现根据 `challenge` 自动计算口令的函数。实验涉及寄存器传参、整数位运算、循环、分支、递归、链表和函数指针。

2 实验环境与要求

- 操作系统：Windows，编译、调试和测试在 WSL Ubuntu 中完成；
- 运行环境：Linux x86-64，使用课程提供的个人 `bomb`；
- 实验目录：Windows 下为 `E:\ICS \Lab 2`，WSL 中为 `/mnt/e/ICS/Lab2`；
- 工具：GCC、Make、Python 3、objdump、GDB 和仓库提供的公开测试；
- 编译参数：`-std=c11 -O2 -Wall -Wextra -Wpedantic -Werror`；
- 个人仓库：`https://github.com/wuxiangjie84/ics-2026-bomblab`。

六个常规关卡的求解函数根据 64 位 `challenge` 输出若干无符号十进制整数；Honor 还需要使用 `context` 计算口令，输出为一整行文本。所有求解函数都通过 `cap` 限定输出缓冲区的容量。求解实现填写在 `defuser.c` 中，函数签名、接口和原有驱动保持不变。

3 实验过程

先用 objdump 查看各关及其辅助函数的指令，遇到跳转表、节点表和函数指针表时，同时核对表中的实际地址顺序。再用 GDB 观察函数参数、返回值和局部对象中的字段，将指令转换成 C 中可计算的关系。实现后使用原有 Makefile 编译，先分关测试，再运行完整七关：

```
cd /mnt/e/ICS/Lab2
make
python3 public_test.py
python3 public_test.py --secret
```

调试单关时，`./defuser --seed 42 --phase 1` 生成口令，`./bomb --seed 42 --phase 1` 负责检查，二者使用相同的 seed 和关卡编号。除公开测试外，还检查了不同分支、32 位整数回绕和输出容量。在这些测试中没有发现计算或输出错误。结果统一放在第 5 节。

4 各关的分析与实现

下文将当前关卡的 `challenge` 记成 C，并约定：

```
L = (uint32_t)C;
H = (uint32_t)(C >> 32);
M = (uint32_t)(C >> 16);
```

L 是低 32 位，H 是高 32 位，M 是右移 16 位后保留的低 32 位。计算式采用 C 语法：^ 是按位异或，>> 是无符号右移。32 位加、乘和左移都只保留低 32 位。

4.1 第一关：寄存器传参与算术运算

第一关读入一个无符号整数。`phase_01` 调用 `p1_compute` 计算目标值，再将其返回值与输入比较。分析时需要确定三个辅助函数的参数及返回值之间的关系。

`p1_compute` 通过移位和寄存器复制提取 L, H, M。`p1_step0` 接收 C, H，`p1_step1` 接收 H, M，`p1_step2` 接收第二个函数的返回值与 L。关键指令如下：

```
# p1_step0
xor $0x4421,%esi
lea (%rsi,%rdi,8),%eax
# after p1_step0 returns
mov %eax,%ecx
# p1_step1
lea (%rdi,%rdi,2),%eax
add %esi,%eax
xor $0xf294,%eax
# p1_step2
lea (%rdi,%rsi,8),%eax
xor $0x5284,%eax
# at the end of p1_compute
xor %ecx,%eax
```

`ecx` 保存 `p1_step0` 的返回值。后两个辅助函数更新 `eax`，其指令均未改写 `ecx`。因此，末尾的异或将第三个函数的返回值与第一个函数的返回值合并。

恢复出的关系是：

```
t0 = (H ^ 0x4421) + 8 * L;
t1 = (3 * H + M) ^ 0xf294;
t2 = (t1 + 8 * L) ^ 0x5284;
answer = t2 ^ t0;
```

`lea` 在这里用来计算加法和乘法，并不读取括号所表示地址中的内容。`p1_step0` 虽然用 64 位寄存器参与计算，结果却写进 32 位的 `eax`，因此用 L 计算 `8 * L` 并保留低 32 位，与原指令的结果一致。

4.2 第二关：循环与递推关系

第二关要求六个无符号整数，检查过程由首项计算和后续五项的循环计算组成。首项对应的指令为：

```
shr $0x1f,%rax
movzbl %al,%eax
add $0x18,%eax
```

这里 `0x1f` 是 31，`movzbl` 只取右移结果的低 8 位，所以第一个数是 `((C >> 31) & 255) + 24`。

循环中 `edx` 从 1 开始，是当前数组下标；`ecx` 从 11 开始，每轮加 2，决定右移位数。`esi` 保存前一项，乘以 5 后参与本轮计算：

```
lea 0xb(%rdx),%edi
lea (%rsi,%rsi,4),%esi
shr %cl,%rax
and $0x1f,%eax
xor %edi,%eax
add %esi,%eax
# each iteration
add $0x1,%rdx
add $0x2,%ecx
```

因此可以直接按数组顺序生成六个数：

```
a[0] = ((C >> 31) & 255) + 24;
for (i = 1; i < 6; ++i) {
    part = (C >> (11 + 2 * (i - 1))) & 31;
    a[i] = 5 * a[i - 1] + (part ^ (i + 11));
}
```

各项按下标顺序递推生成。循环使用的移位量依次为 11、13、15、17、19；每轮先计算异或项，再与前一项的 5 倍相加。

4.3 第三关：跳转表与分支选择

这一关输入两个整数。第一个数 `s = C & 7` 决定分支，第二个数由对应分支计算。传入分支函数的数据是：

```
s = C & 7;
x = (uint32_t)(C >> 3) ^ H;
```

`p3_dispatch` 中出现了下面这条间接跳转：

```
cmp $0x7,%eax
ja  0x402242
notrack jmp *0x4057e0(,%rax,8)
```

跳转表的每项为 8 字节地址，访问偏移为下标的 8 倍。表项下标与函数名中的编号并不一致。例如，下标 1 对应地址 `0x402220`，该地址的跳转指令指向 `p3_case_4`。据此恢复出的分支对应关系如下。

表 1：第三关跳转表与计算关系

s	实际函数	第二个整数的计算式
0	p3_case_0	3*x + 0x6655
1	p3_case_4	3*x + 0xb4f5
2	p3_case_3	5*x + 0x4c7e8
3	p3_case_1	(x ^ 0xd9d4) + (x >> 1)
4	p3_case_5	5*x + 0x32217
5	p3_case_7	(x ^ 0xd56a) + (x >> 2)
6	p3_case_6	(4*x) ^ 0xd3ce
7	p3_case_2	(4*x) ^ 0x14ca

实现使用 `switch` 保留跳转表的实际顺序，输出分支编号 `s` 及对应的计算结果。

4.4 第四关：递归调用与返回值计算

第四关要求一个整数。它先得到目标值和辅助参数 `salt`，再调用 `p4_walk`：

```
target = C & 127;
salt = (uint32_t)(C >> 7) ^ H;
answer = walk(target, 0, 127, salt);
```

`p4_walk` 按二分方式定位目标，并在递归返回时逐层计算返回值。每层的中点是 `mid = low + ((high - low) >> 1)`。目标值等于中点时结束递归，否则缩小区间，等子调用返回后再乘以 3、异或。

分支方向要根据比较和跳转判断。`eax` 是目标值，`edi` 是中点：

```
cmp %edi,%eax
jb  0x402310
# return from the larger-target branch
0x402306: xor $0x8,%eax
# return from the smaller-target branch
0x402324: xor $0x1a,%eax
```

`jb` 是无符号“小于就跳转”，所以跳到 `0x402310` 的是目标较小的分支。整个递归关系为：

```
mid = low + ((high - low) >> 1);
if (target == mid)
    return (mid + 0x6f) ^ salt;
if (target < mid) {
    child = walk(target, low, mid - 1, salt);
    return (3 * child) ^ (mid + salt) ^ 0x1a;
}
child = walk(target, mid + 1, high, salt);
return (3 * child) ^ (mid + salt) ^ 0x08;
```

例如目标是 0 时，区间中点依次是 63、31、15、7、3、1、0，命中 0 后还要沿调用栈逐层计算返回值。目标已经限制在 0 到 127，所以区间持续缩小后能够命中。较小目标分支使用常量 `0x1a`，较大目标分支使用常量 `0x08`。

4.5 第五关：结构体布局与链表遍历

这一关输出两个数：累计值，以及最后一个访问节点的 tag。根据访存偏移可恢复节点布局：偏移 0 是 8 字节 next 指针，偏移 8 是 4 字节 tag，偏移 12 是 4 字节 weight，所以每个节点占 16 字节。

节点表从 `0x405b60` 开始。下表按内存顺序用十进制编号；tag 和 weight 用十六进制。next 一列已经把地址换成节点编号：

表 2: 第五关节点布局与链接关系

节点	next	tag	weight	节点	next	tag	weight
0	9	`067e`	`088c`	6	10	`0752`	`7b3c`
1	4	`040e`	`2f81`	7	3	`036f`	`d6eb`
2	0	`0886`	`e1de`	8	NULL	`0982`	`f515`
3	6	`0134`	`72ac`	9	5	`0546`	`8284`
4	7	`0a55`	`4bbe`	10	8	`0254`	`600d`
5	1	`0b18`	`14ac`	11	2	`006d`	`a214`

头指针表 `0x405b20` 的八项对应 `11, 2, 0, 9, 5, 1, 4, 7`。因此访问哪个节点由 next 决定，不是每次把内存地址加 16。实现中保存这些固定结构数据，用数组下标代替原程序的地址，计算过程如下：

```
node = heads[C & 7];
length = 1 + ((C >> 3) & 3);
total = (uint32_t)(C >> 6) ^ H ^ 0x2cb0;
for (i = 0; i < length; ++i) {
    total = ((total + nodes[node].weight) * 5) ^ nodes[node].tag;
    last_tag = nodes[node].tag;
    node = nodes[node].next;
}
```

首节点也要参与计算。`p5_traverse` 在进入后续循环前就执行了：

```
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
lea (%rax,%rax,4),%eax
xor %esi,%eax
```

这已经加入了首节点的 weight 和 tag。后面的循环才沿 next 前进。所以访问长度是 1 时，也要算一次首节点；长度是 4 时，一共处理四个节点。输出依次为累计值 `total` 和末节点的 `last_tag`。

4.6 第六关：函数指针表与间接调用

第六关通过函数指针表选择运算函数，并连续执行两次间接调用。两次调用之间的数据传递如下：

```
call *0x405dc0(,%rax,8)
xor  $0x6703,%eax
mov  %eax,%edi
call *0x405dc0(%rbx)
add  $0x634,%eax
```

第一处用下标乘 8 访问表项。第二处的 `rbx` 已经是 `C & 0x38`，本身就是字节偏移，换成数组下标时要除以 8。两次调用组成：

```
x = (uint32_t)(C >> 6) ^ H;
i = C & 7;
j = (C >> 3) & 7;
t = op[i](x) ^ 0x6703;
answer = op[j](t) + 0x634;
```

函数表中存在重复表项，其下标与函数的对应关系如下：

表 3: 第六关函数指针表与运算关系

下标	实际函数	对输入 x 的操作
0	p6_op_3	ROR32(x, 5)
1	p6_op_2	(x + 0xc2bf) ^ (x >> 1)
2	p6_op_4	ROL32(x, 5) ^ 0x43a4
3	p6_op_5	(x ^ 0xe859) + (x << 7)
4	p6_op_1	((x << 7) + 0xd083) ^ (3*x)
5	p6_op_4	ROL32(x, 5) ^ 0x43a4
6	p6_op_0	(x ^ (x >> 2)) + 0x6529
7	p6_op_5	(x ^ 0xe859) + (x << 7)

rol、ror 是 32 位循环移位，移出的位会从另一端补回来。对应代码是 ROL32(x,5) = (x << 5) | (x >> 27) 和 ROR32(x,5) = (x >> 5) | (x << 27)。实现通过两个方向的移位与按位或组合，保留循环移位的全部位。

4.7 Honor：输入布局与字段计算

Honor 的输入是一整行文本，前面还有标记字段。它先由 secret_copy 把整行复制到局部对象，再从对象里的固定位置取数据：

```
lea  -0xd0(%rbp),%r12
call secret_copy
cmpl $0x4e45504f,-0xb0(%rbp)
lea  -0xac(%rbp),%rdi
call secret_phase
```

对象起点是 rbp-0xd0，标记在 rbp-0xb0，相差 32 字节；数字字符串从 rbp-0xac 开始，也就是起点之后 36 字节。0x4e45504f 在小端内存中依次是 O、P、E、N，所以行的布局为：

```
[32 bytes][OPEN][a b c]
```

Honor 同时检查字段偏移与数值。标记和数字字符串须位于 gate 指定的位置。本实现用 32 个 A 填充，OPEN 后直接接第一个数字。整行最长 59 个字符，低于 gate 允许的 163 字节上限。输出时使用 snprintf 限制写入长度，并根据返回值判断内容是否被截断。

secret_phase 给格式检查函数传入的数量是 3，且实际使用了第三个参数 context。它由原有驱动根据前六关口令计算，用 Q 表示。首先，r12 保存 challenge，r13 保存 context。第一字段对应下面的检查：

```
mov %r13d,%ecx
and $0x3,%ecx
cmp %ecx,-0x2c(%rbp)
```

因此第一个数 a 必须等于 context 的低两位。随后分别将 challenge、context 的高低 32 位异或，得到 k 和 h。secret_mix 的参数是 k ^ h，其三条运算指令为：

```
rol $0x3,%eax
add $0xe0dc,%eax
xor $0x7927,%eax
```

这得到下面的 mix。返回后，r9d 保存 (a * 0xe0dc) ^ (uint32_t)(Q >> 16)，它与 mix 异或，再由 cmp %eax,-0x28(%rbp) 检查第二个数 b。最后用 lea 和 add 得到 h + a + 0x7927 + (k >> 3)，与 b 异或后检查第三个数 c。三个字段的计算关系为：

```
k = (uint32_t)C ^ (uint32_t)(C >> 32);
h = (uint32_t)Q ^ (uint32_t)(Q >> 32);
a = (uint32_t)Q & 3;
mix = (ROL32(k ^ h, 3) + 0xe0dc) ^ 0x7927;
b = mix ^ (a * 0xe0dc) ^ (uint32_t)(Q >> 16);
c = b ^ (h + a + 0x7927 + (k >> 3));
```

secret_mix 中的 rol $0x3,%eax 对应这里的循环左移 3 位。输出依次为 a、b、c，其中后两个字段同时依赖 challenge 和 context。在后文记录的 seed 下，GDB 观察到的字段值如下：

```
challenge = 0x149c79a40db85d55
context   = 0x1aa48e1fb72acd5b
offset 32: 79 'O' 80 'P' 69 'E' 78 'N'
offset 36: "3 726233616 2616865564"
```

5 测试结果与验证

5.1 公开测试与补充测试

执行 make 编译后，六个常规关卡的公开测试共通过 60/60，Honor 的公开测试通过 10/10。分关结果见图 1 至图 4。

补充测试使用边界值、伪随机 challenge 及公开列表之外的 seed，由真实的 defuser 生成口令，再交给 bomb 检查。伪随机源固定为 20261015。测试结果如下：

表 4: 补充测试范围与结果

检查范围	实际结果
每关 689 个 challenge，包含边界值及随机值	六关共 4134 次，全部通过
261 个不在公开列表中的 seed，六关与 Honor 串联	261/261
第三关分支；第四关目标值	8/8；128/128，全部覆盖
第五关起点和长度；第六关两次函数选择	32/32；64/64，全部覆盖
Honor 第一字段的四种取值	4/4，全部覆盖
输出容量检查（同时启用未定义行为检查器）	5376/5376

689 个 challenge 包括 0 到 255、全 1、交替位、单比特及相邻边界值，以及 256 个伪随机值，去重后逐个检查。单关测试通过 `--challenge` 直接指定题目值；串联测试通过 `--seed` 生成六关及 Honor 的题目值，并将七行口令一起交给 `bomb` 检查。

输出容量测试独立调用七个求解函数。每组生成一对 `challenge` 和 `context`，共 128 组；每个函数测试六种容量，因此共检查 $128 \times 7 \times 6 = 5376$ 次。令 n 为口令的字节数（不含字符串末尾的空字符），六种容量为 $0, 1, n-1, n, n+1, 256$。检查内容包括返回值、成功时的字符串内容及缓冲区两侧的保护字节。其中 $n+1$ 才能完整容纳口令和末尾的空字符。运行时启用 GCC 未定义行为检查器，未报告错误。

补充测试脚本随报告源码一并提供，位于 `verification` 目录。在 WSL 中进入解压后的报告源码目录，执行：

```
python3 verification/verify_all.py /mnt/e/ICS/Lab2
python3 verification/verify_buffers.py /mnt/e/ICS/Lab2
```

测试记录保存在 `verification/results` 子目录中。

```
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ pwd
/mnt/e/ICS/Lab2
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ make
make: Nothing to be done for 'all'.
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ exit
exit
```

图 1: 第一、二关公开测试结果（各 10/10）

```
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ pwd
/mnt/e/ICS/Lab2
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ exit
exit
```

图 2: 第三、四关公开测试结果（各 10/10）

```
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ pwd
/mnt/e/ICS/Lab2
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ exit
exit
```

图 3: 第五、六关公开测试结果（各 10/10）

```
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ pwd
/mnt/e/ICS/Lab2
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ exit
exit
```

图 4: Honor 公开测试结果（10/10）

5.2 公开列表之外的 seed 验证

选取 `seed=3141592653589793238`，该值不在 `tests/public.json` 中。六个常规关卡及 Honor 的 challenge 和口令记录如下，均经 `bomb` 检查通过。

表 5: 所选 seed 下的 challenge 与口令

关卡	本次 challenge	实际口令
1	`0xaa8beb14f9461213`	`3226777063`
2	`0xc2669cd89afa7019`	`201 1007 5065 25351 126773 633880`
3	`0xe5a80828eca5498a`	`2 3644058981`
4	`0x751dc0c487b174ff`	`268141372`
5	`0x9c4d9a2b7679fbe9`	`3616691510 1662`
6	`0x3b531994c94d110b`	`4064833088`
Honor	`0x149c79a40db85d55`	32 个 `A`，后接 `OPEN3 726233616 2616865564`

完整七关的验证命令如下，运行结果见图 5。

```
cd /mnt/e/ICS/Lab2
make
{
    ./defuser --seed 3141592653589793238
    ./defuser --secret --seed 3141592653589793238
} | ./bomb --seed 3141592653589793238 | cat
```

`--secret` 生成 Honor 的输入行，两次调用使用相同的 seed。末尾的 `cat` 原样转发输出。终端中的 `Secret phase defused.` 和 `Secret public tests` 对应 Honor。

```
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ pwd
/mnt/e/ICS/Lab2
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ {
> ./defuser --seed 3141592653589793238
> ./defuser --secret --seed 3141592653589793238
> } | ./bomb --seed 3141592653589793238 | cat
Phase 1 challenge: 0xaa8beb14f9461213
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0xc2669cd89afa7019
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0xe5a80828eca5498a
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0x751dc0c487b174ff
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x9c4d9a2b7679fbe9
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0x3b531994c94d110b
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
wuxiangjie@wuxiangjie:/mnt/e/ICS/Lab2$ exit
exit
```

图 5: 所选 seed 下的完整七关运行结果

6 实验小结

这次实验中，读懂单条汇编指令只是第一步，还要把参数传递、计算过程和返回值连起来看。例如第一关中，第一个辅助函数的结果保存在 `ecx`，后面的计算结束后还要用到它；第四关找到目标值后，也要沿递归调用逐层计算返回值。

第三关和第六关需要按表中实际保存的地址确定函数顺序，不能直接照函数名中的编号排列。第五关则要沿 `next` 指针访问节点，并把首节点算进去。Honor 除了检查数值，还要求 `OPEN` 和后面的数字出现在指定位置。

7 参考资料与工具使用

[1] 计算机系统基础课程网站：Lab2 BombLab 实验说明。`https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/`。

[2] 课程 BombLab 仓库：README。

objdump 用于反汇编与静态数据检查，GDB 用于观察寄存器和内存，GCC、Make 与 Python 用于编译及测试，LaTeX 用于报告排版。Codex 用于辅助汇编分析、代码检查优化，以及报告润色。